

WEEK 2 | CONTAINERIZED AI SYSTEMS

SESSION 1 OF 4

Docker Fundamentals for GPU-Enabled AI Workloads

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India



What you will be able to do after this session

- Explain the layered image model and why it matters for multi-gigabyte AI images.
- Distinguish image vs container, Dockerfile vs build context, layer vs tag.
- Describe the CUDA driver / toolkit split and the failure modes it causes.
- Run a GPU-backed container and verify passthrough with nvidia-smi.
- Build a pinned, GPU-capable image from a Dockerfile — hands-on.

The reproducibility problem in AI

1 A real research stack

Python 3.12 + CUDA 13.0 + cuDNN 9
+ PyTorch 2.11 + a pinned
transformers + system libs (libgl1,
ffmpeg) + custom CUDA kernels
needing a specific gcc.

2 Ship it to a collaborator...

Different driver, different Python,
different BLAS. It breaks — silently or
loudly. Conda envs help with Python
but not with system libs, drivers, or
kernels.

Containers freeze the entire userspace — OS libs, Python, frameworks — into one shippable artifact.

Image vs container vs layer — the mental model

Image

Immutable, layered filesystem + metadata. Built once, run anywhere a compatible runtime exists.

Container

A running (or stopped) instance of an image with its own writable layer, namespaces, and cgroups.

Layer

One filesystem diff per Dockerfile step. Cached and shared between images — vital when bases are 5–10 GB.

All images are huge. Layer caching and ordering are not nice-to-haves — they decide whether rebuilds take 30 seconds or 30 minutes.

Anatomy of a GPU Dockerfile

```
FROM nvidia/cuda:13.0.0-runtime-ubuntu24.04    # no driver inside
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.12 python3-pip git && rm -rf /var/lib/apt/lists/*
COPY requirements.txt /tmp/requirements.txt      # deps first
RUN pip install --no-cache-dir -r /tmp/requirements.txt
COPY . /workspace                              # code last
WORKDIR /workspace
CMD ["python3", "train.py"]
```

- Base image supplies the CUDA userspace; the host supplies the driver.
- Dependencies before code: editing train.py must not re-install PyTorch.

The CUDA split: driver vs toolkit

On the HOST

Kernel module + GPU driver (e.g. 580.xx). Installed once per machine. Never inside the image.

In the IMAGE

CUDA toolkit + libraries (cuBLAS, cuDNN, NCCL) + framework. Ships with your container.

- Compatibility rule: driver version must be new enough for the toolkit in the image.
- Symptom of mismatch: 'CUDA driver version is insufficient for CUDA runtime version'.
- Fix direction: upgrade the host driver, or pick an older CUDA base image.

How the GPU reaches your container

- 1 **docker run --gpus all ... triggers the NVIDIA runtime hook**
- 2 **The hook injects device nodes (/dev/nvidia*) and the host driver libraries**
- 3 **Inside, CUDA apps see the GPU as if installed natively — no driver in the image**

Selective exposure: `--gpus '"device=0,1"'` or
`NVIDIA_VISIBLE_DEVICES=0,1`

Choosing a base image deliberately

base

Bare CUDA runtime API. Smallest. You add everything.

runtime

+ cuBLAS, cuDNN, NCCL libraries. Right default for running frameworks.

devel

+ nvcc and headers. Needed only to compile kernels / build wheels.

- Framework images (e.g. pytorch/pytorch, NGC) trade size for convenience and tested combos.
- Production pattern: build with devel, deploy on runtime (multi-stage — Session 2).

Lab 1 — Verify GPU passthrough

```
# 1. Host sanity: driver loaded?
nvidia-smi

# 2. Toolkit installed? (one-time per machine)
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# 3. The moment of truth: GPU visible INSIDE a container
docker run --rm --gpus all \
    nvidia/cuda:13.0.0-base-ubuntu24.04 nvidia-smi

# 4. Limit exposure to one GPU
docker run --rm --gpus '"device=0"' \
    nvidia/cuda:13.0.0-base-ubuntu24.04 nvidia-smi
```

Success = the same GPU table inside the container as on the host.

Lab 2 — Build and run your first GPU image

```
# Dockerfile
FROM nvidia/cuda:13.0.0-runtime-ubuntu24.04
RUN apt-get update && apt-get install -y python3.12 python3-pip \
    && rm -rf /var/lib/apt/lists/*
RUN pip install --no-cache-dir torch==2.11.* \
    --index-url https://download.pytorch.org/whl/cu130
COPY check_gpu.py /app/check_gpu.py
CMD ["python3", "/app/check_gpu.py"]

# check_gpu.py
import torch
print('cuda available:', torch.cuda.is_available())
print('device:', torch.cuda.get_device_name(0))

# Build + run
$ docker build -t gpu-hello .
$ docker run --rm --gpus all gpu-hello
```

Layer caching: 30-second vs 30-minute rebuilds

Anti-pattern

COPY . /app first, then pip install.
Any code edit invalidates the install layer — full dependency rebuild on every commit.

Pattern

COPY requirements first, install, then COPY the code. Code edits reuse every cached layer above.
Pair with BuildKit cache mounts for pip.

```
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt
```

Common failure modes and first responses

- 'driver version is insufficient' → host driver older than image toolkit. Check `nvidia-smi` vs image tag.
- 'could not select device driver "nvidia"' → container toolkit missing or Docker not restarted.
- `torch.cuda.is_available()` is False but `nvidia-smi` works → wheel built for a different CUDA; reinstall from the matching index.
- DataLoader crashes with 'bus error' → `/dev/shm` too small; run with `--shm-size=8g` (more in Session 2).
- Image works on machine A, OOMs on machine B → different GPU memory; verify with `nvidia-smi` before blaming code.

Exercise & check yourself

Exercise

Containerize one of your own scripts: pinned base image, deps-before-code layering, and a CMD that proves the GPU is visible. Measure rebuild time after a code-only edit.

Check-for-understanding

1. Why is the GPU driver never installed inside the image?
2. What does the CUDA version in `nvidia-smi` actually report?
3. Why does copying `requirements.txt` before the code speed up rebuilds?

Recap & what's next

- Containers freeze userspace; the host keeps the kernel and the GPU driver.
- The NVIDIA Container Toolkit injects devices + driver libs at run time.
- Base-image tags (base / runtime / devel) are a deliberate size-vs-capability choice.
- Layer ordering is the lever that keeps AI image rebuilds fast.
- **Next — Session 2: deterministic environments, dependency pinning, and a containerized Jupyter workflow.**